Bug / Error Log

Bug / Error Log

Records the errors, suboptimal points, or lack of rigor produced by OpenCode (and the various free models) during the assignment, along with how they were fixed. The "3 code suggestions" section of the report will select the 3 most representative entries from this table.

- Only entries marked `[CODE]` count as the required "3 code suggestions".
- Entries marked `[EDA/process]` are supplementary material and do not count toward those 3.
- Current qualifying count: **[CODE] 3 / 3** (Step 1 passed independent verification with no errors and is not counted).

#	Stage	Problem type	Original content (summary)	Problem description	Lecture basis	Fix	After fix
1	[EDA/process] Step0 EDA	Factual error (wrong)	Report said "the maximum spectral power peak ≈23.8h, then 11.9h"	The main/secondary spectral peaks were swapped. Measured, the maximum peak is at 11.9h (power 7.65e8), and 23.8h is the secondary peak (5.77e8)	Spectral/periodicity analysis (LSTM/Transformer modeling of periodic structure in sequences)	Recompute the periodogram and rewrite the conclusion by power ranking	Conclusion changed to "semi-daily 12h dominant, daily 24h secondary"
2	[EDA/process] Step0 EDA	Numerical error (wrong)	Report said "ACF lag=144 ≈0.147"	Value miscopied; measured 0.128	Autocorrelation is the basic statistic for sequence modeling (RNN/LSTM rely on history)	Recompute the ACF and correct it	lag=144 ≈0.128, lag=288 ≈0.169
3	[EDA/process] Step0 EDA	Weak argument (suboptimal)	Report claimed "ACF peaks fall at integer multiples of 24h" and cited lag=436≈0.164	The 24h multiple is 432, not 436; ACF at lag=432 is only 0.089, not a peak. The evidence does not hold	Sequence periodicity and autocorrelation analysis	Remove the lag=436 argument, keep only the two real peaks at 144/288, and honestly note the data-length limitation	Argument now consistent with measurements
5	[CODE] Step2 LSTM	Overfitting / suboptimal (wrong)	Training script had no validation set and no early stopping, trained a fixed 50 epochs and directly took the last-epoch model	test loss rose monotonically from 1.13 to 4.66 (severe overfitting); the final model RMSE=282 Wh, 3.3× worse than the naive baseline 86.6 Wh, i.e. "trained a model worse than a trivial baseline"	LSTM is structurally complex and costly to train (slide P30: longer training time, higher computational complexity), so a validation set/early stopping must be used to control	Cut a validation set from the end of the train segment by time, save and fall back to weights by best val loss; also compare	With v2 "validation set + early stopping": multivariate RMSE=157.5 (still worse than naive 86.6), univariate RMSE=78.8 (better than naive); univariate finally chosen

#	Stage	Problem type	Original content (summary)	Problem description	Lecture basis	Fix	After fix
					training instead of blindly running out epochs	univariate input	
6	[CODE] Step4 generalizable pipeline	Missing feature selection / suboptimal (wrong)	The generalizable script run_forecast.py fed all 28 numeric columns (including the already-noted "uninformative" random columns rv1/rv2) into the model without selection, and provided no feature/input selection option	On both time periods the model was worse than naive: release week LSTM 157.5 / Transformer 108.9 (naive 86.6); periodB LSTM 17.5 / Transformer 16.2 (naive 11.0). Feeding uninformative features aggravates overfitting and distribution shift	The lecture emphasizes that sequence modeling must capture effective historical information (RNN/LSTM remember dependencies via the hidden state, P16-17); redundant/noisy features dilute the signal, and the model-efficiency chapters also advocate compressing useless information	Add `--features {all,univariate}`: `all` drops rv1/rv2 by default (26 columns), `univariate` uses only the target history	**Applied and accepted**: release week univariate LSTM RMSE 157.5→**78.8**, Transformer 108.9→**77.1**, both beating naive 86.6; periodB univariate LSTM 22.6→**12.5**, Transformer 21.0→**13.2** (naive 11.0, still slightly negative, a distribution shift). Feature count self-evidenced by n_params (28→26→1 corresponds to 57409→56897→50497)
7	[CODE] Step2 LSTM	test-set leakage / wrong	The training loop computed the loss on the **test set** every epoch (`test_loss = criterion(model(X_test_t), y_test_t)`, train_lstm.py:110-112), and used this `test_losses` curve as the training monitoring/early-stopping basis for plotting (train_lstm.py:162-163); finally it directly took the	Using the test set for per-epoch monitoring and model selection is **test-set leakage**: the test set is no longer "unseen data", and its loss is used to decide the training process. Even looking without selecting already contaminates the evaluation; here it even directly took the last-epoch weights, leading to RMSE=282 (3.3×	Train/validation/test three-way split and early stopping: model selection must use a **validation set**, and the test set is evaluated only once at the end (corresponding to the course early stopping / validation chapters, and P30 on using a validation set to control LSTM training cost)	Change to cutting a validation set from the end of the train segment by time, using only val loss per epoch for monitoring and best-weight saving, with the test set used only for final evaluation	From v2 onward the test set is fully isolated; univariate RMSE=78.8 (better than naive 86.6); `train_lstm_v2.py` / `transformer_forecast.py` / `run_forecast.py` all follow this protocol

#	Stage	Problem type	Original content (summary)	Problem description	Lecture basis	Fix	After fix
			last-epoch weights (train_lstm.py:119)	worse than naive 86.6)			

Notes

- Entries 1 and 2 are **factual/numerical errors** generated by AI that require manually re-checking the raw data to find — showing that EDA reports must be reconciled entry by entry and cannot be taken on trust.
- Step 1 (data processing) passed independent recomputation verification with **no data leakage, no window out-of-bounds, no look-ahead**; no errors found.
- Entry 5 is the first qualifying `[CODE]` item: the core lesson is "finishing training ≠ a usable model"; a **validation set** must be used to select the model, and comparing against the naive baseline is what reveals that the model is actually worse.
- Entry 7 and entry 5 are both in `train_lstm.py` but are **two independent defects**: entry 5 is "no validation set / no early stopping", entry 7 is "using the test set as the monitoring signal (test-set leakage)". They were fixed together, but the code locations (`train_lstm.py:110-112,119,162-163`) and knowledge points (test leakage) differ, so they are listed separately.
- Qualifying `[CODE]` entries: `#5` (no validation/early stop overfitting), `#6` (generalizable pipeline without feature selection), `#7` (test-set leakage), count **3 / 3**.
- Number `#4` is vacant: it was originally an `/init` process item (involving the development-assistant file `AGENTS.md`); since it is out of submission scope it was removed from this table, with the number kept to stay consistent with report references.